



3.1 Implementing Conditional Edges in LangGraph



Introduction to Conditional Edges

- **What are Conditional Edges?**
 - Edges that route execution to different nodes based on state evaluation
 - Enable dynamic decision-making in agent workflows
 - Allow agents to adapt behavior based on runtime conditions
- **Use Cases:**
 - Tool selection based on query type
 - Quality checks that determine next steps
 - Retry logic when errors occur
 - Different processing paths for different data types

Anatomy of a Conditional Edge

- **Three Core Components**
 - **Source Node:** Where the edge originates
 - **Routing Function:** Evaluates state and returns next node name
 - **Path Map:** Dictionary mapping return values to node names
- **Routing Flexibility**
 - Not limited to binary (2-way) decisions
 - Can route to 3+ different nodes based on logic

Basic Structure

```
workflow.add_conditional_edges(  
    "source_node",          # Source  
    routing_function,       # Router  
    {                        # Path map  
        "option_a": "node_a",  
        "option_b": "node_b"  
    }  
)
```

- **Flow**

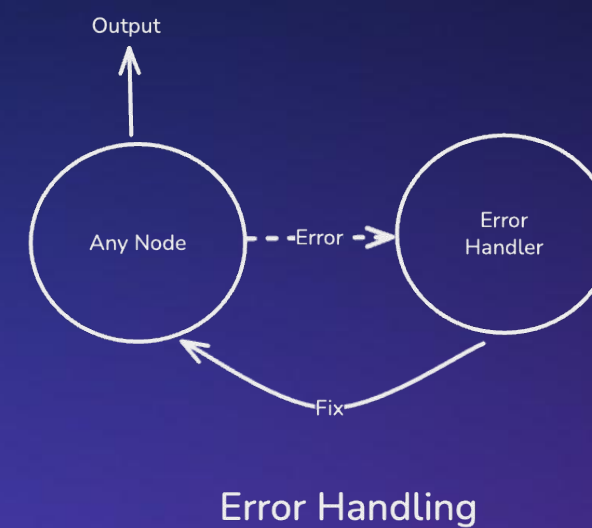
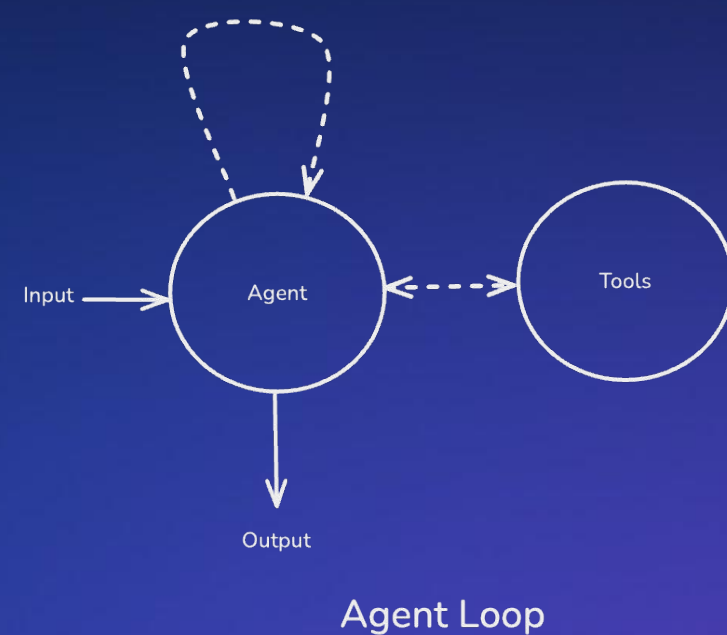
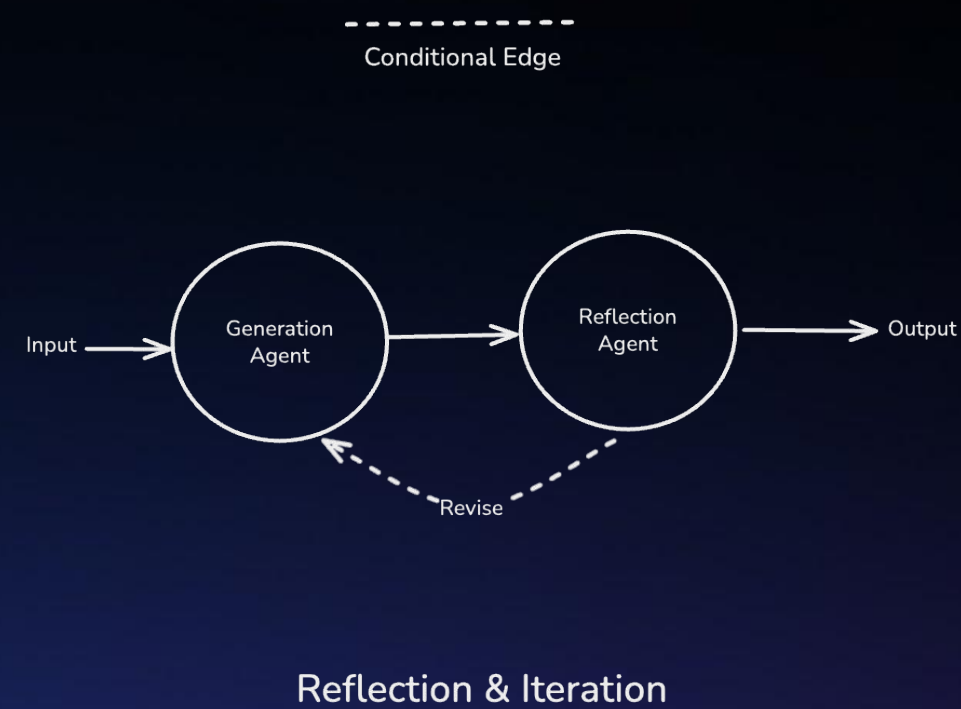
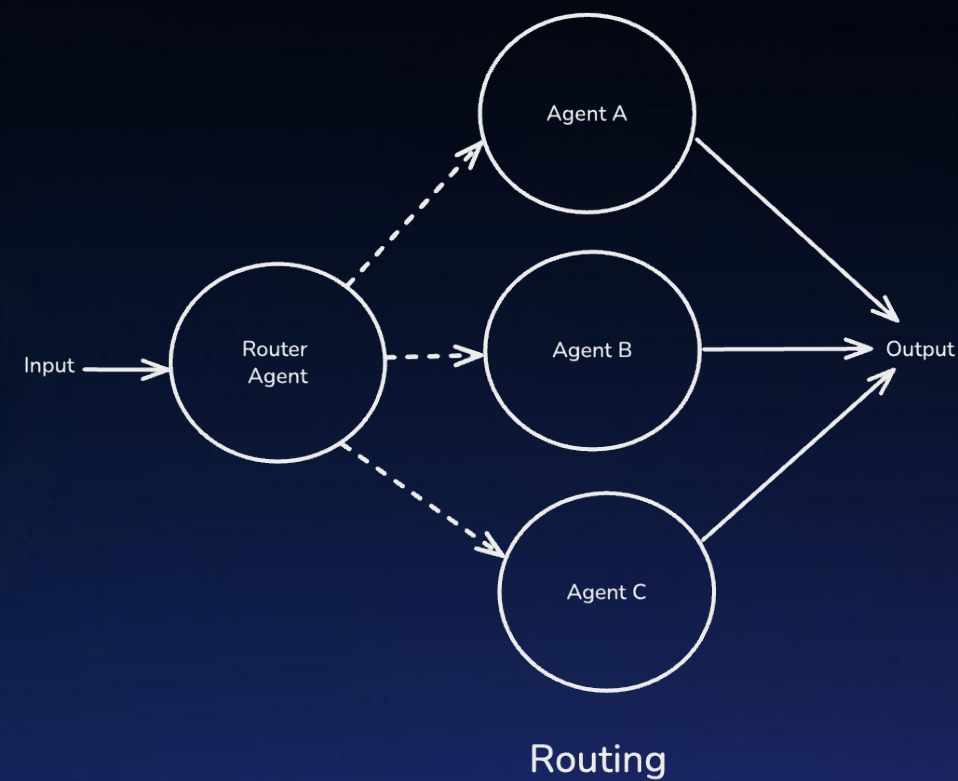
- Source node executes → Updates state → Routing function evaluates state → Determines next node

Creating Routing Functions

- **Routing Function Requirements**
 - Must accept state as parameter
 - Must return a string indicating the next node
- **Simple Routing Function**

```
def route_by_condition(state: AgentState) -> str:  
    if state["needs_tool"]:  
        return "use_tool"  
    else:  
        return "respond_directly"
```

Common Patterns with Conditional Edges



Best Practices

- **Routing Function Design**
 - Keep routing logic simple and readable
 - Avoid complex computations in routing functions
- **State Management**
 - Ensure state contains all data needed for routing
 - Consider adding routing metadata to state
- **Path Mapping**
 - Use descriptive names for paths
 - Ensure all possible return values are mapped
- **Testing**
 - Test each routing path independently

O'REILLY®

3.2 Designing Custom Workflows with State Graphs



From Components to Workflows

- **Workflow Composition**

- Combining state, nodes, and edges to build agentic systems
- State flows through nodes, edges control execution path
- Each node transforms state, edges route to next step

- **Why Custom Workflows?**

- Explicit control over agent behavior and decision points
- Modular design enables testing and iteration per component
- Different patterns for different problem types

Linear Workflow Pattern

- **When to Use**

- Fixed sequence of processing steps
- Each step depends on previous step's output
- Input progressively refined through stages

- **Architecture**

- $\text{START} \rightarrow (\text{Node A}) \rightarrow (\text{Node B}) \rightarrow (\text{Node C}) \rightarrow \text{END}$
- State accumulates information at each stage
- Each stage adds value or transforms data

Linear Workflow Pattern: Example Code

```
workflow = StateGraph(ReportState)

# Multi-stage transformation
workflow.add_node("fetch_data", fetch_data_node)
workflow.add_node("analyze", analyze_data_node)
workflow.add_node("create_charts", create_charts_node)
workflow.add_node("write_report", write_report_node)

# Connect sequentially
workflow.add_edge(START, "fetch_data")
workflow.add_edge("fetch_data", "analyze")
workflow.add_edge("analyze", "create_charts")
workflow.add_edge("create_charts", "write_report")
workflow.add_edge("write_report", END)
```

Cyclic Workflow Pattern

- **When to Use**
 - Iterative refinement needed
 - Quality checks with retry logic
 - Agent needs multiple attempts to succeed
- **Architecture**
 - Loops back to previous nodes based on conditions
 - State tracks iteration count and progress
- **Key Components**
 - Router function determines continue or exit
 - Max iterations to prevent infinite loops
 - State accumulates context across iterations

Cyclic Workflow Pattern: Example Code

```

# Node can loop back to itself or continue
workflow.add_node("generate", generate_node)
workflow.add_node("validate", validate_node)

workflow.add_edge(START, "generate")
workflow.add_edge("generate", "validate")

# validate_node can route back to generate or END
workflow.add_conditional_edges(
    "validate",
    route_after_validation, # Returns "generate" or "end"
    {"generate": "generate", "end": END}
)
```

Branching Workflow Pattern

- **When to Use**

- Different processing paths for different inputs
- Specialized nodes for specific scenarios
- Dynamic routing based on state analysis

- **Architecture**

- Router node examines state and selects path
- Multiple specialized processing branches
- Paths may converge or end independently

Branching: Example Code for Routing

```
def route_by_intent(state: AgentState) -> str:
    """Route to different nodes based on user intent"""
    intent = state["user_intent"]

    if intent == "query":
        return "search_node"
    elif intent == "create":
        return "generation_node"
    elif intent == "analyze":
        return "analysis_node"
    else:
        return "fallback_node"
```

Workflow Design Best Practices

- **Start with Workflow Map**
 - Sketch out the flow and design the state structure before coding
 - Identify decision points, and loops
- **Modular Node Design**
 - Each node handles one responsibility
 - Nodes should be testable independently
- **State Management**
 - Include workflow control fields (iteration counts, phase tracking, error tracking)
 - Use reducers for accumulating data (lists, aggregations)
- **Error Handling**
 - Add validation nodes at key points
 - Include fallback paths for failures

O'REILLY®

3.3 Debugging Agents by Analyzing State Transitions



Observe State During Execution

Agent behavior emerges from state transformations, so real-time visibility into changes to the state can help with debugging.

Two complementary approaches:

- **Custom logging:** Add targeted logs at critical points
 - Node entry/exit
 - Routing decisions
 - Tool invocations
 - State validation
- **Use for domain-specific insights, business logic validation**
- **Framework Capabilities:** Use LangGraph's built-in capabilities, pick your granularity:
 - Stream full state after each update
 - Stream only changes from each node
 - Stream detailed execution metadata
- **General debugging, understanding execution flow**

Streaming Modes in LangGraph

- **Different perspectives on execution:**
 - **values mode:** Complete state snapshot after each node
 - See full picture at each step
 - Useful for understanding accumulated state
 - **updates mode:** Only what changed in each node
 - Focus on node outputs
 - Reduces noise, highlights transformations
 - **debug mode:** Execution metadata and internals
 - Framework-level insights
 - Performance and routing information

O'REILLY®

3.4 Enabling Tool Interoperability with MCP



The Integration Problem

- **Before MCP: The M x N Problem**
 - Connecting M AI applications to N tools requires M x N custom integrations
 - Each integration must be built from scratch
 - Results in duplicated effort across teams and organizations
- **Real-world impact**
 - Teams can't specialize (AI work vs. integration work)
 - Schema variations between different LLM providers
 - Tight coupling between AI features and tool integrations
 - Organizational bottlenecks slowing AI adoption

What is MCP?

- Model Context Protocol (MCP) is an open standard that defines a uniform way for AI models to access external data and tools.

The USB-C Analogy:

- Before USB-C: Different chargers for every device
- After USB-C: One standard, universal compatibility
- MCP does the same for AI tool integrations

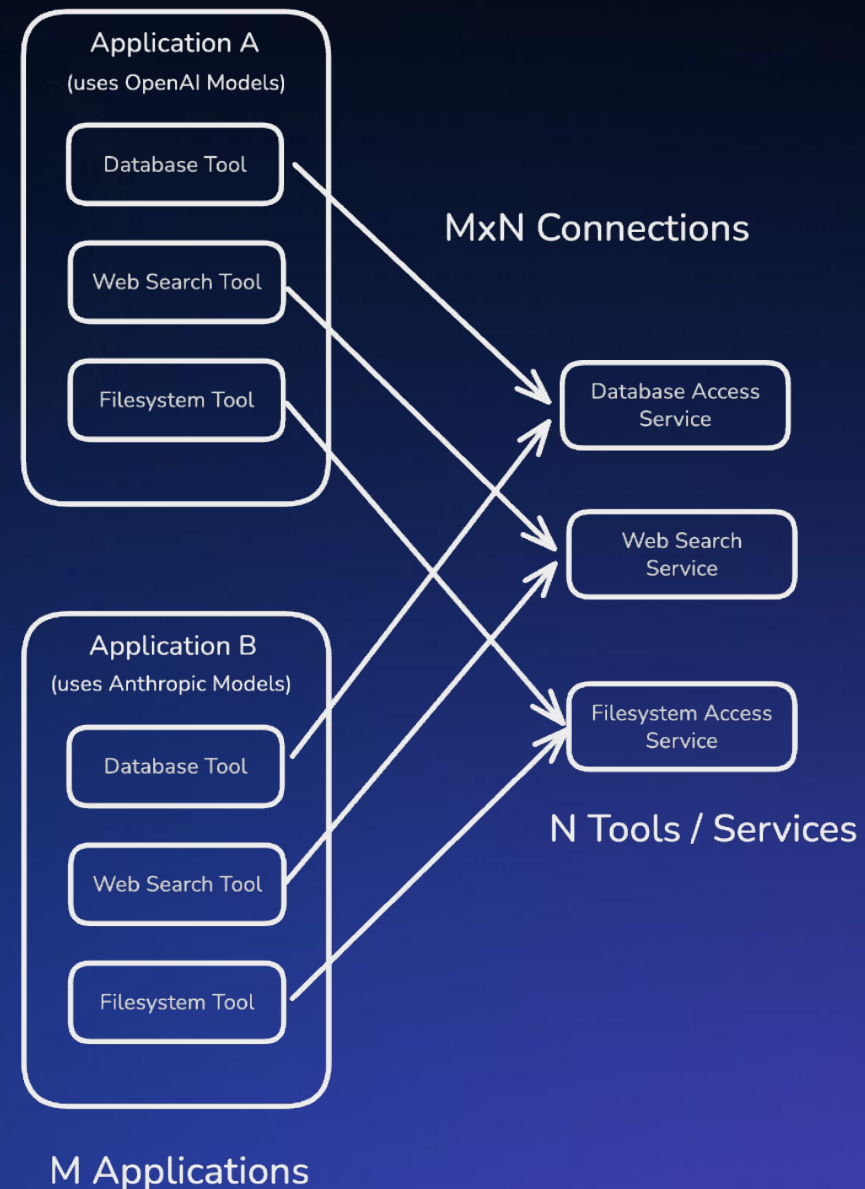
Key benefit: Transforms $M \times N$ integrations into $M + N$ relationships

- Write once, use everywhere - reusable community servers
- Foundation model provider agnostic
- Any MCP-compatible host can use it

From MxN to M+N Integrations

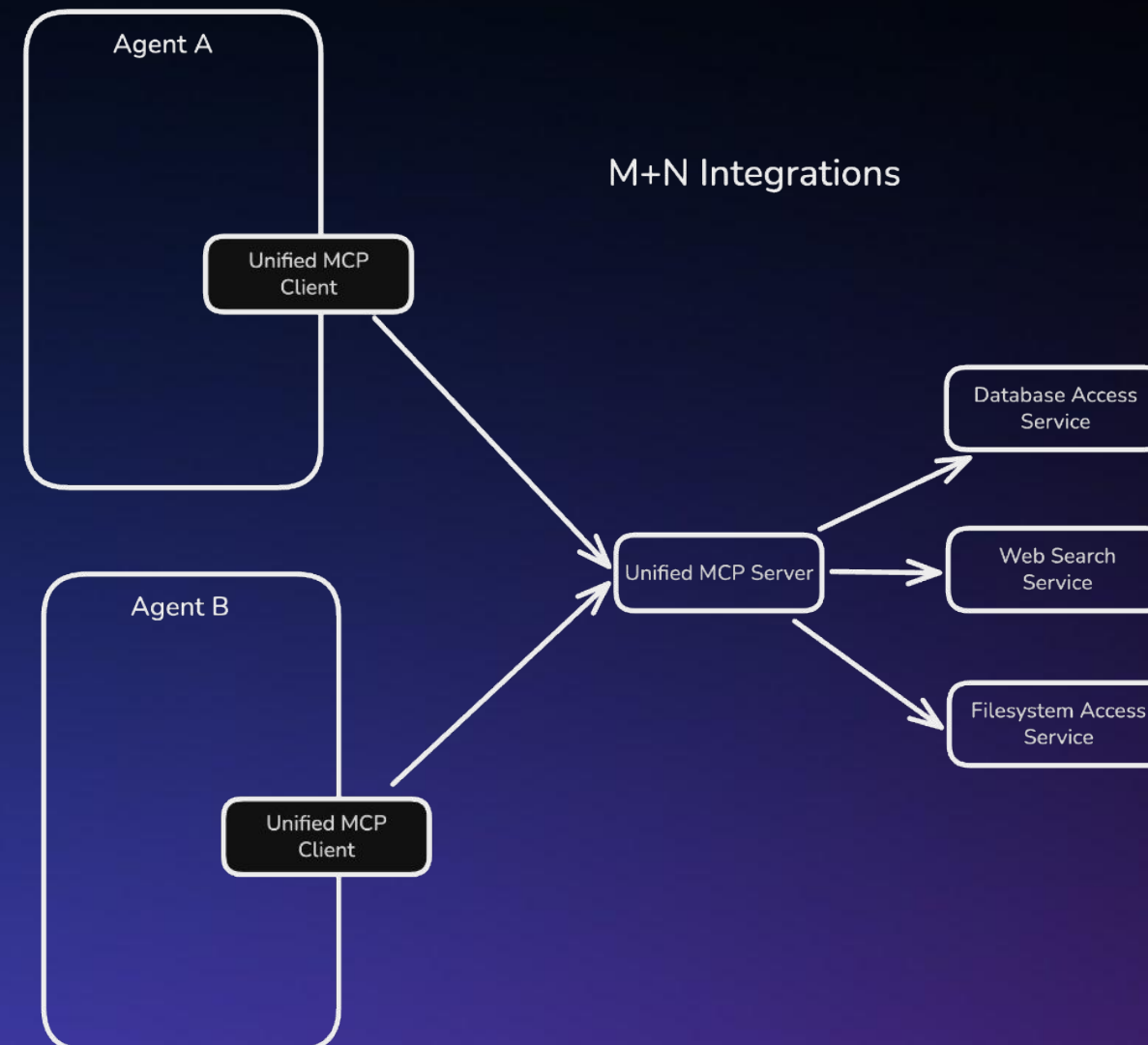
Without MCP

- Write custom tools, and interface to underlying service for each application
- AI Applications and Tools are tightly coupled



With MCP

- Write MCP servers once, or use official / community created servers
- Agent and tools are decoupled
- MCP clients can be simply configured in the agent, with native support in most Agent SDKs



Core Components of MCP

- **Hosts**

- LLM applications that users interact with
- Examples: Claude Desktop, IDEs (Cursor, VS Code), Custom AI agents
- Initiate connections to MCP servers

- **MCP Clients**

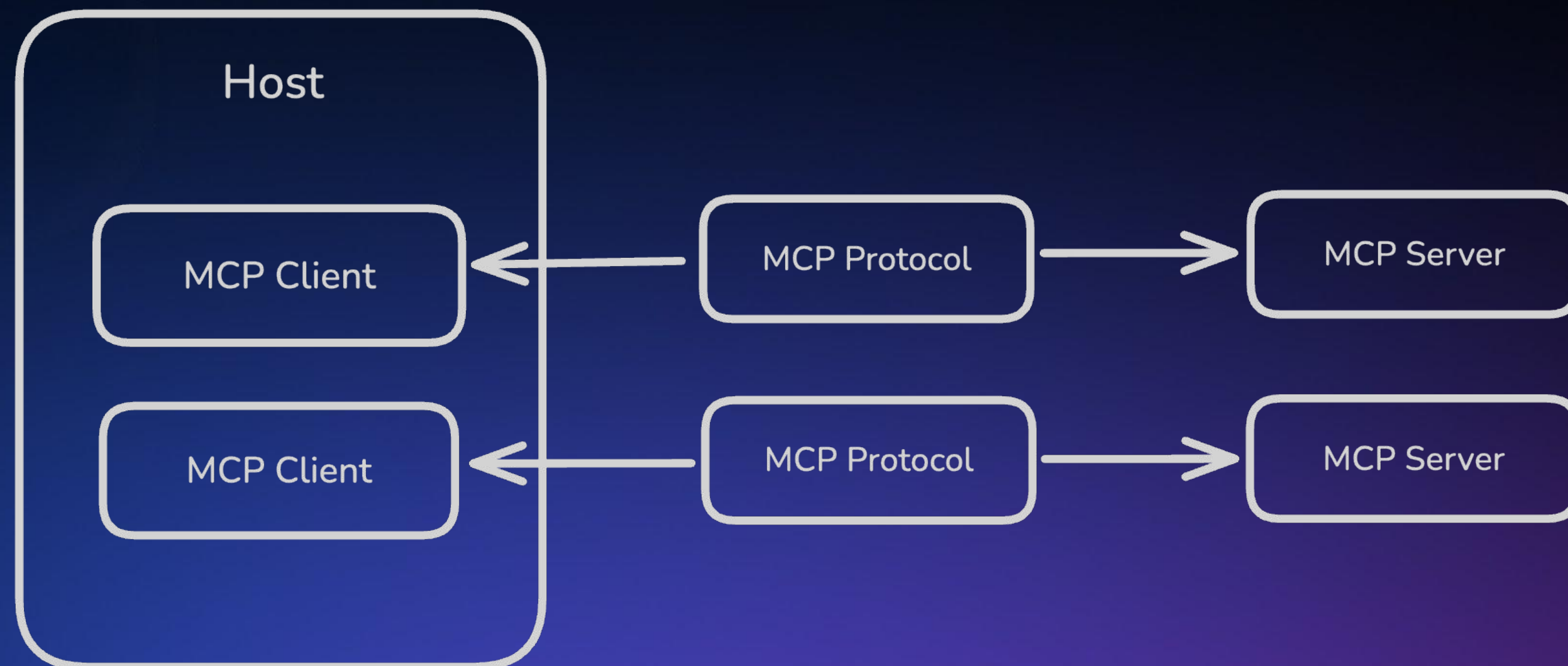
- Protocol handlers embedded within hosts
- Manage connections to one or more MCP servers
- Handle message routing and capability negotiation

- **MCP Servers**

- Programs that expose specific capabilities
- Connect to external systems (databases, APIs, file systems)
- Communicate through standardized MCP interface

MCP Architecture

MCP Client-Server Architecture

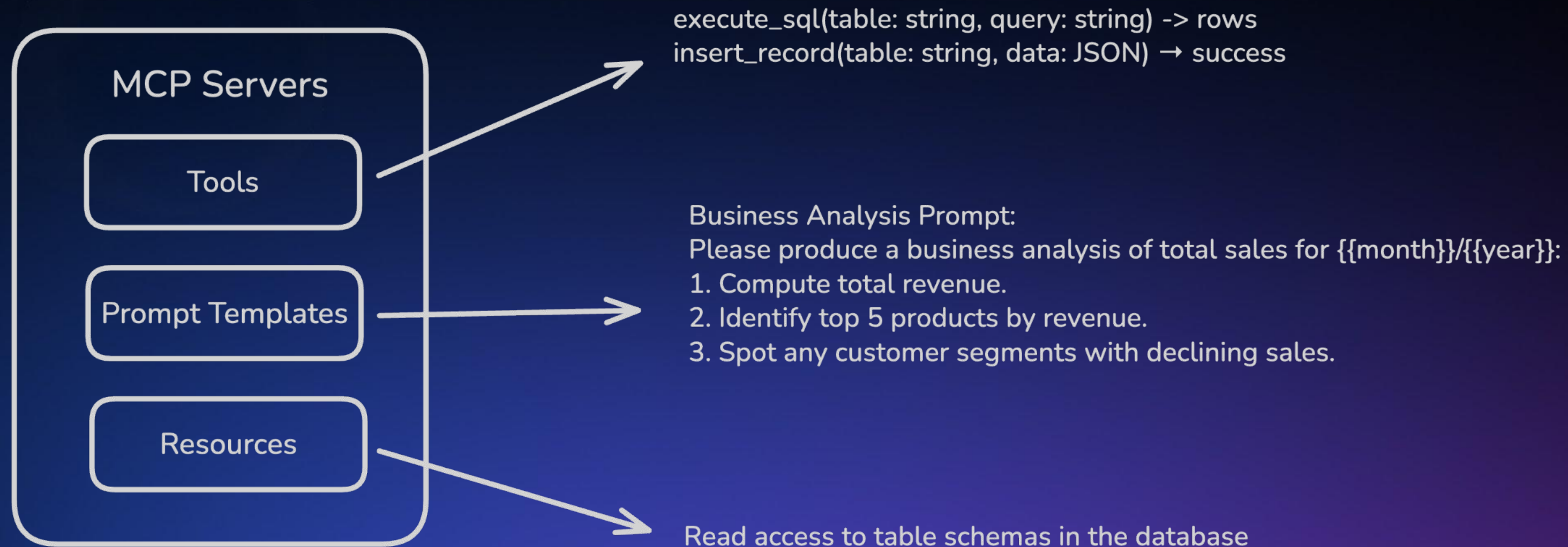


What MCP Servers Expose

- **Tools**
 - Invokable functions the AI can call
 - Examples: `search_files`, `run_query`, `send_message`
 - Enable AI agents to take actions in external systems
- **Resources**
 - Read-only data access
 - Examples: File contents, database records, API responses
 - Provide context without side effects
- **Prompt Template**
 - Pre-defined interaction patterns
 - Examples: Reusable templates for common tasks
 - Ensure consistent, effective prompting

Example MCP Server

Example for a MCP Server Serving Data from a SQL DB



What MCP Clients Provide

- **Roots**

- Define URI boundaries for server operations
- Limit scope of what servers can access
- Security and sandboxing mechanism
- Example: Restrict file access to specific directories

- **Sampling**

- Allow servers to request LLM inference from the host
- Enables agentic patterns within servers
- Servers can ask the host's LLM to process information
- Useful for complex server-side logic

Transport Mechanisms

- **Stdio (Standard Input/Output)**
 - For local deployments
 - Server runs as subprocess of host
 - Simple, low-latency communication
 - Best for: Desktop apps, local development
- **Streamable HTTP**
 - For remote deployments
 - Replaces older SSE transport
 - Supports stateless operations
 - Best for: Cloud-hosted servers, production
- **Security: OAuth 2.1**
 - For HTTP-based transports with PKCE support for secure authentication

The MCP Ecosystem

- **Popular Server Categories:**
 - **Databases:** PostgreSQL, MongoDB, Redis, SQLite
 - **Developer Tools:** GitHub, GitLab, Jira, Linear
 - **Cloud Services:** AWS, GCP, Azure
 - **Communication:** Slack, Discord, Email
 - **File Systems:** Local files, Google Drive, S3
- **Who's Using MCP:**
 - Anthropic (Claude Desktop, Claude Code)
 - OpenAI (adopted the standard)
 - Various IDE makers (Cursor, Windsurf, etc.)
 - Enterprise AI applications

When to Use MCP

- **Good fit for MCP**
 - Building AI applications that need multiple tool integrations
 - Creating reusable tool servers for your organization
 - Extending existing AI apps with new capabilities
 - Multi-agent systems needing shared tool access
- **Consider alternatives when:**
 - Single, simple tool integration (tool with direct API call may be simpler)
 - Highly specialized, one-off use case
 - Performance-critical, low-latency requirements where protocol overhead matters

Common Pitfall: Confusing MCP with Direct Tool Calling

MCP is a protocol for tool interoperability, not a replacement for tool calling itself.

Your agent still uses tool calling to invoke functions.

MCP standardizes how those tools are discovered, described, and connected.

Think of it as the plumbing that connects your agent to tools, not the tools themselves.

O'REILLY®

3.5 Building a Basic RAG System for Agents

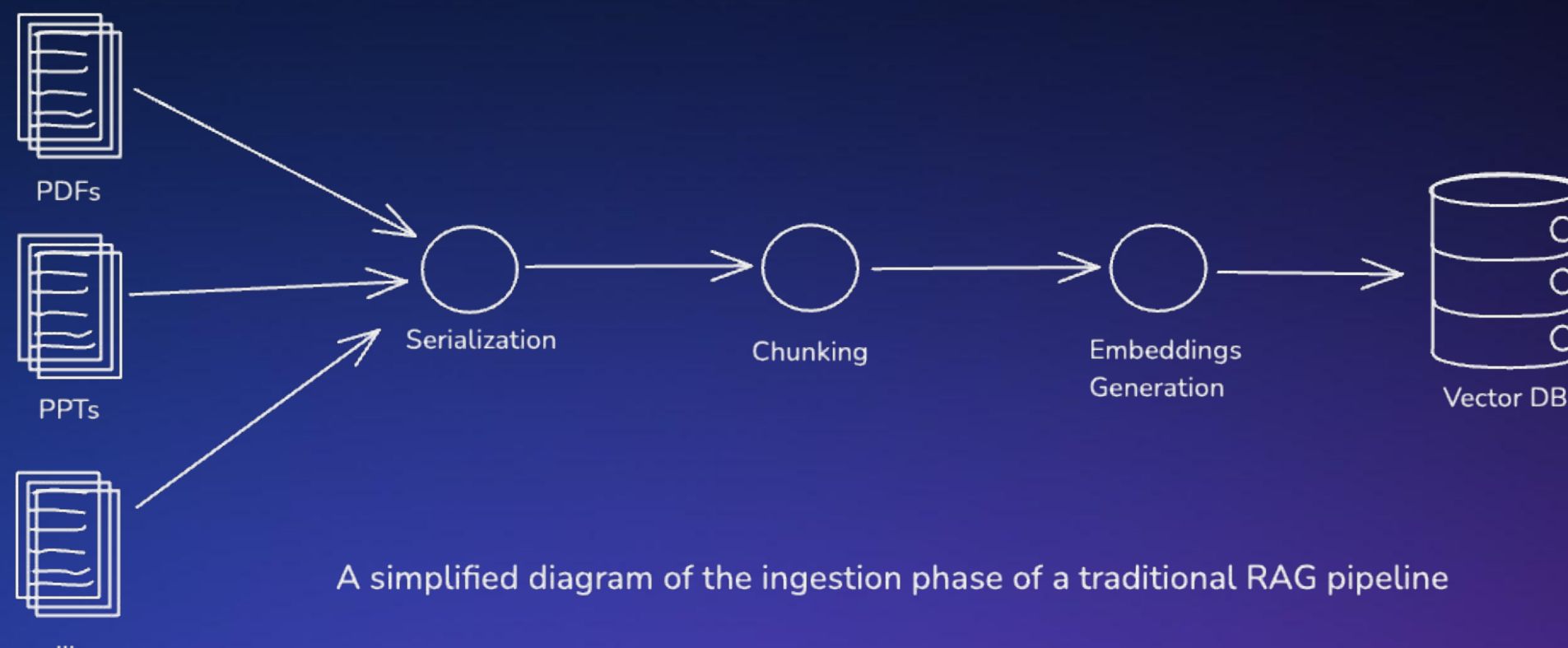


Two Phase Architecture for Retrieval

- Phase 1: Indexing
- Phase 2: Query

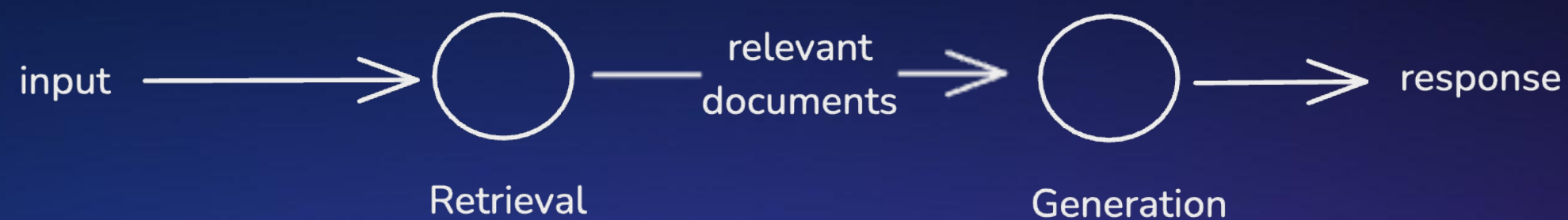
Phase 1: Indexing Overview

- Happens once or periodically when documents are updated
- Converts knowledge base into searchable vector representations
- Prepares for similarity search



Phase 2: Query Overview

- Happens at runtime for each user query
- Finds relevant context and augments LLM prompt
- Returns grounded, factual responses



A simplified diagram of the inference phase of a traditional RAG pipeline

Key RAG Components

- **Embeddings:** Convert text to vectors that capture semantic meaning
- **Vector Store:** Efficiently stores and searches document embeddings
- **Retriever:** Finds most relevant chunks for a given query
- **Generator (LLM):** Creates responses grounded in retrieved context

Indexing Steps

- **Step 1: Loading**

- Load documents from various sources (PDFs, text files, databases); can include serialization

- **Step 2: Chunking**

- Split large documents into smaller chunks
- **Why?** LLMs have context limits; smaller chunks improve retrieval precision
- **Chunk size:** Depends on use case, start with 500-1500 tokens (balance between context and specificity) with some overlap across boundaries

Indexing Steps

- **Step 3: Generate Embeddings**

- Convert each text chunk into a vector (embedding)
- **What are embeddings?** Numerical representations that capture semantic meaning. Similar content → Similar vectors
- Example: "dog" and "puppy" have close vectors; "dog" and "car" are distant

- **Step 4: Store in Vector Database**

- Store embeddings with original text and metadata
- Vector DB optimized for similarity search (finds "nearby" vectors fast)

Query Steps

- **Step 1: Embed the Query**

- User asks: "What is the company's vacation policy?"
- Convert query to embedding vector (same embedding model as indexing)
- **Why same model?** Ensures query and documents are in the same vector space

- **Step 2: Similarity Search**

- Vector DB compares query embedding to all document embeddings
- Finds top-k most similar chunks (typically k=3-5)
- **How?** Measures distance (cosine similarity, euclidean distance)
- **Returns:** most relevant text chunks + their metadata

Query Steps

- **Step 3: Augment LLM Context**
 - Take retrieved chunks and format as context
 - Create prompt: Example: "Use this context: [chunks] to answer: [query]"
 - Critical: Retrieved context is included in the prompt
- **Step 4: Generate Grounded Response**
 - LLM generates response using retrieved context
 - Response is grounded in your knowledge base
 - Avoids hallucination by referencing specific documents

RAG in LangGraph Workflows

- RAG with Nodes and Edges
- Conceptual Workflow: START → (Retrieve Node) → (Generate Node) → END
- Agent State Structure

```
class AgentState(TypedDict):  
    query: str          # User's question  
    context: str        # Retrieved document chunks  
    response: str       # Final LLM response
```

Common Pitfall: Picking the Wrong Chunking Technique

- Engineers often pick arbitrary chunk sizes (500 tokens, 1000 tokens) without testing whether the right information actually gets retrieved.
- **Common failures**
 - **Chunks too small:** Important context split across chunks, neither is retrieved
 - **Chunks too large:** Relevant sentence buried in irrelevant text, similarity score drops
 - **No overlap:** Key information at chunk boundaries gets lost
- The right chunking technique depends on the use case. Before building the full pipeline, test retrieval with 10 real queries for your use case. If the right chunks are not retrieved, your RAG will fail no matter how good your prompt is.

O'REILLY®

3.6 Constructing Plan-and-Execute Agent Systems





Introduction

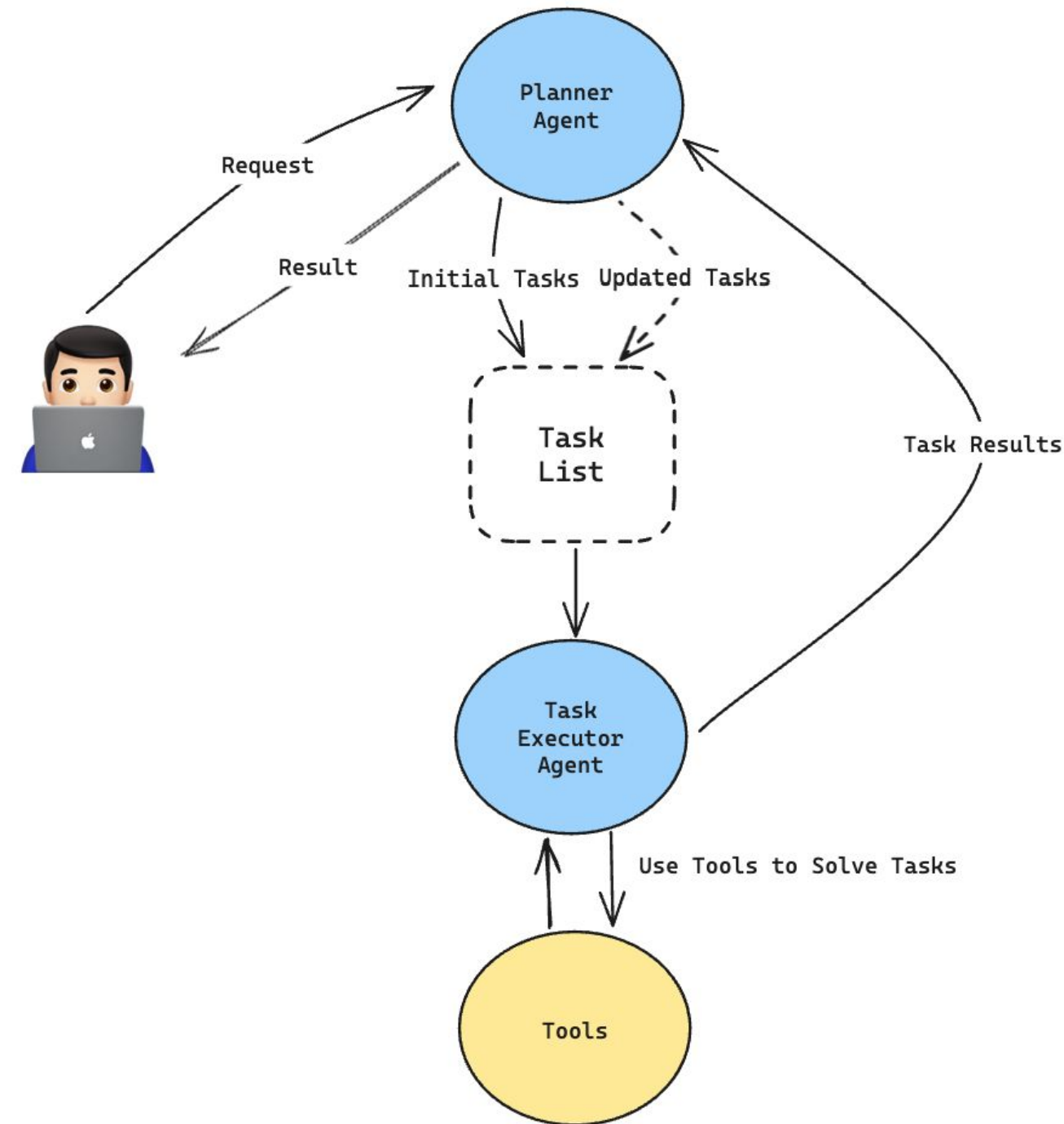
- **Objective**
 - Review the *Planning* architectural pattern for AI agents.
 - Learn how to design and implement a Plan-and-Execute system using LangGraph.
- **Core Idea**
 - **Planning Phase:** Develop a multi-step plan to achieve a goal.
 - **Execution Phase:** Follow the plan step-by-step, revisiting and refining as needed.



Plan-and-Execute

- **Comparison with ReAct:**
 - **ReAct:** Think and act one step at a time.
 - **Plan-and-Execute:** Develop an overarching plan and execute it sequentially.
- **Advantages:**
 - **Explicit Long-Term Planning**
 - Enhances the ability to handle complex, multi-step tasks.
 - **Model Efficiency**
 - Utilize smaller models for execution while reserving larger models for planning.

Plan-and-Execute



An Example Planning Workflow

O'REILLY®

3.7 Implementing Deep Planning Agents



What are Deep Agents?

- **Agents You've Seen Till Now:**
 - Simple LLM loop calling tools
 - Handle straightforward, short tasks
 - Basic tool execution without sophisticated coordination
- **Deep Agents:**
 - Advanced architecture beyond basic tool-calling loops
 - Handle complex, long-running tasks
 - Sophisticated coordination and memory management
 - Examples: Claude Code, Deep Research, Manus

Core Characteristics of Deep Agents

- Detailed System Prompts
- Planning Tools
- Sub-agents
- File System Access

Detailed System Prompts

- **Shallow Agent Prompt**
 - Generic instructions: "You are a helpful assistant"
 - Minimal tool guidance
 - No behavioral examples
- **Deep Agent Prompt**
 - Explicit tool usage patterns with decision trees
 - Error handling procedures for common failure modes
 - Multi-step workflows demonstrating best practices
 - Domain expertise embedded in instructions

Detailed System Prompts: Elements

- **Tool Selection Logic:** "When searching code: try function definitions first, then usage examples, then tests"
- **Decision Trees:** "If error contains 'import': check dependencies then verify path then reinstall"
- **Quality Gates:** "Before submitting: run tests, check types, verify no regressions"
- **Fallback Strategies:** "If web search fails: try documentation site, then check cached results, then ask user"

Planning Tools

- **Purpose:** Context engineering to maintain focus
- **Implementation:** Todo lists, step trackers, progress monitors
- **Benefit:** Agent stays organized across long tasks
- **Example:** Claude Code's todo list keeps track of multi-step implementations
- **Key Insight:** Even if functionally no-ops, planning tools serve as prompts to keep agent on track

Sub-agents

- **Task decomposition:** Break complex work into specialized subtasks
- **Focused expertise:** Each sub-agent handles specific domain
- **Context management:** Reduce prompt complexity per agent
- **Example:** Main agent delegates to `code_analyzer`, `test_generator`, `docs_writer`
- **Benefit:** Enables deep focus on individual components while maintaining coordination

File System Access & Memory

- **How File System is Used**

- Persistent memory beyond conversation context
- Shared workspace for multi-agent collaboration
- Long-term state across sessions or restarts
- Artifact storage for intermediate results

- **Key Capabilities**

- Save analysis findings for later reference
- Store plans that multiple sub-agents can access
- Build up knowledge base over extended workflows
- Enable asynchronous agent coordination

Shallow Agents vs Deep Agents

Characteristic	Shallow Agents	Deep Agents
System Prompts	Basic instructions	Detailed with examples
Planning	Minimal/reactive	Explicit planning tools
Architecture	Single agent loop	Multiple specialized sub-agents
Memory	Conversation context, configured long-term memory	File system + persistent storage
Task Complexity	Simple, short tasks	Complex, long-running jobs
Use Cases	Q&A, Simple Lookups	Software Development, Research

Common Pitfall: Deep Agents for Shallow Tasks

Deep agents shine on complex, multi-step research or coding tasks.

For "what's the weather?" they're overkill—adding latency, cost, and complexity.

Match agent sophistication to task complexity.

O'REILLY®

3.8 Adding Human-in-the-Loop Checkpoints

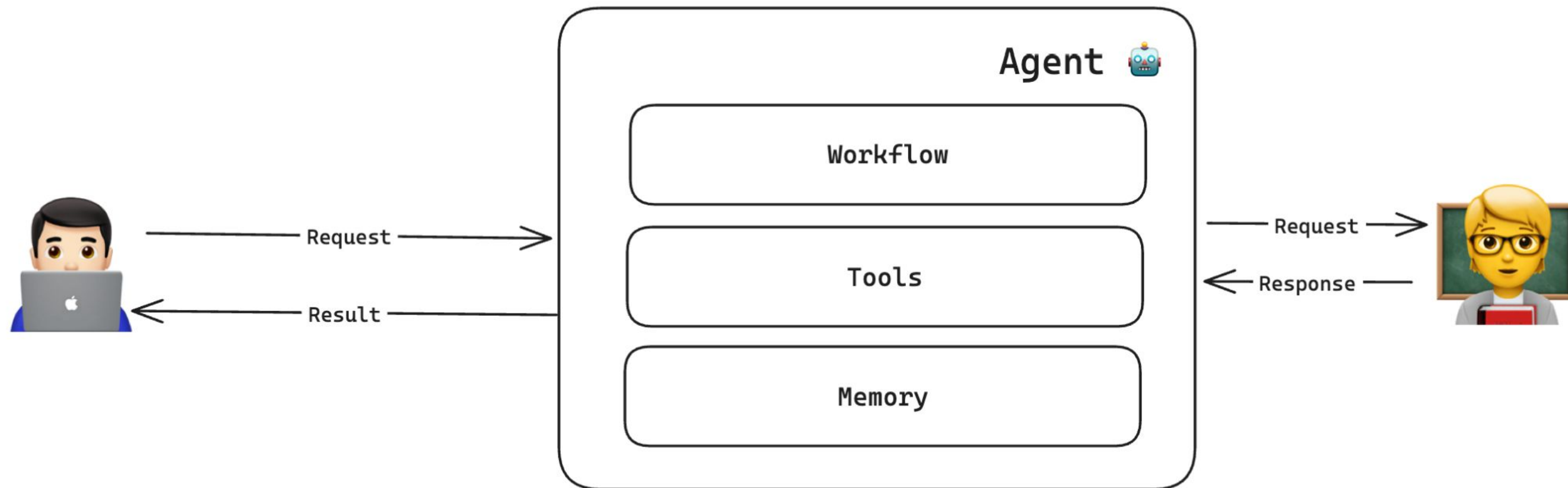




Human Oversight in Agentic Systems

- **Objective:**
 - Implement various human oversight patterns to effectively integrate human interventions within AI workflows.
- **Scenarios:**
 - Approval
 - Edit Agent Actions
 - Wait for Input

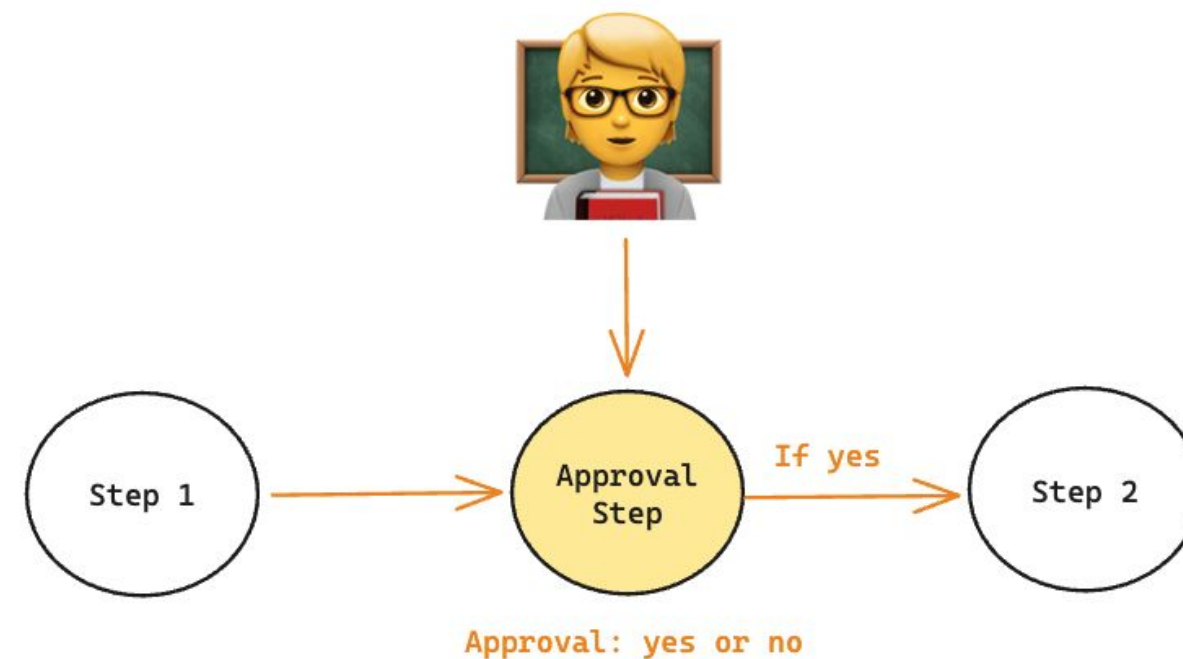
Human Oversight in Agentic Systems



An Example Chatbot Agent with Human-in-the-loop

Approval

- **Description**
 - The AI agent generates outputs or takes actions that require human approval before execution.
- **Use Case**
 - Approving sensitive data writes or critical decision-making steps.



An Example Human-in-the-loop Approval Workflow

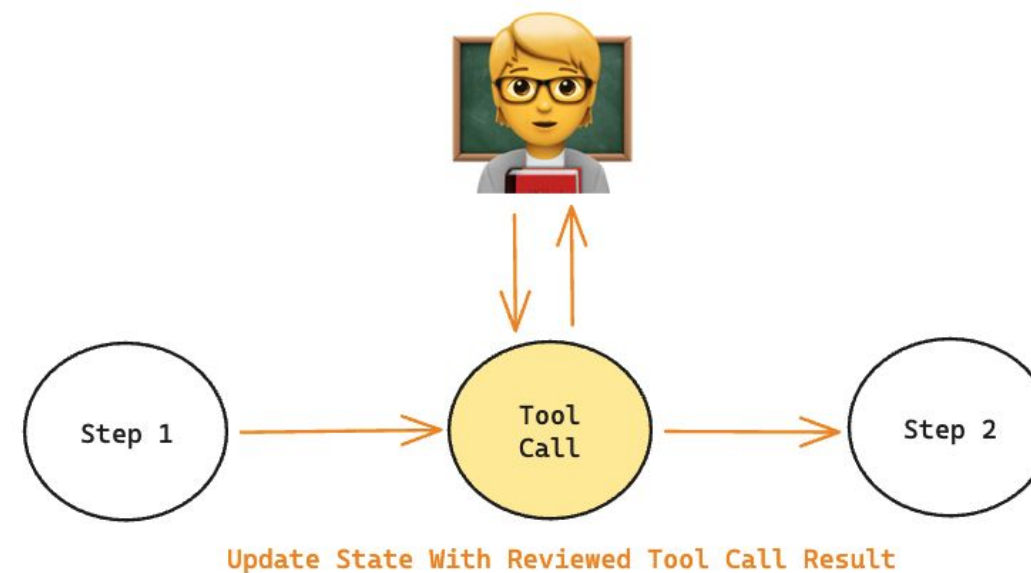
Edit Agent Actions

- **Description**

- Manually updating the graph state is a common human-in-the-loop interaction pattern, allowing the human to edit actions (e.g., what tool is being called or how it is being called).

- **Use Case**

- A human reviewer modifies the parameters of a tool call in a workflow to better suit the current context or to correct an agent's action.



An Example Human-in-the-loop Review Tool Call Workflow

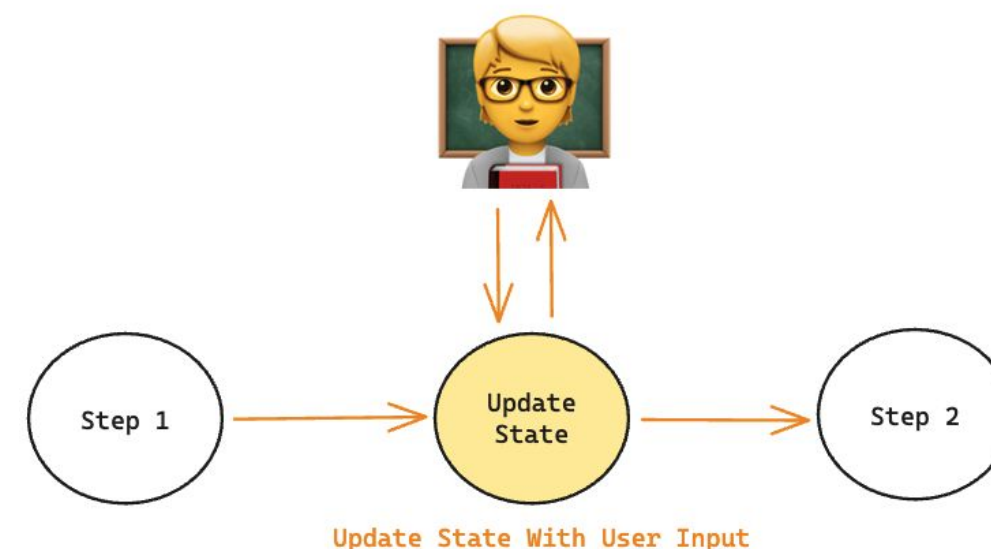
Wait for Input

- **Description**

- Waiting for human input is a common Human-in-the-Loop (HIL) interaction pattern, allowing the agent to ask clarifying questions and await input before proceeding.

- **Use Case**

- A chatbot asks the user for additional details before providing a detailed response, ensuring the information is accurate and relevant.



An Example Human-in-the-loop Wait For Input Workflow

O'REILLY®

3.9 Implementing the Reflection Pattern

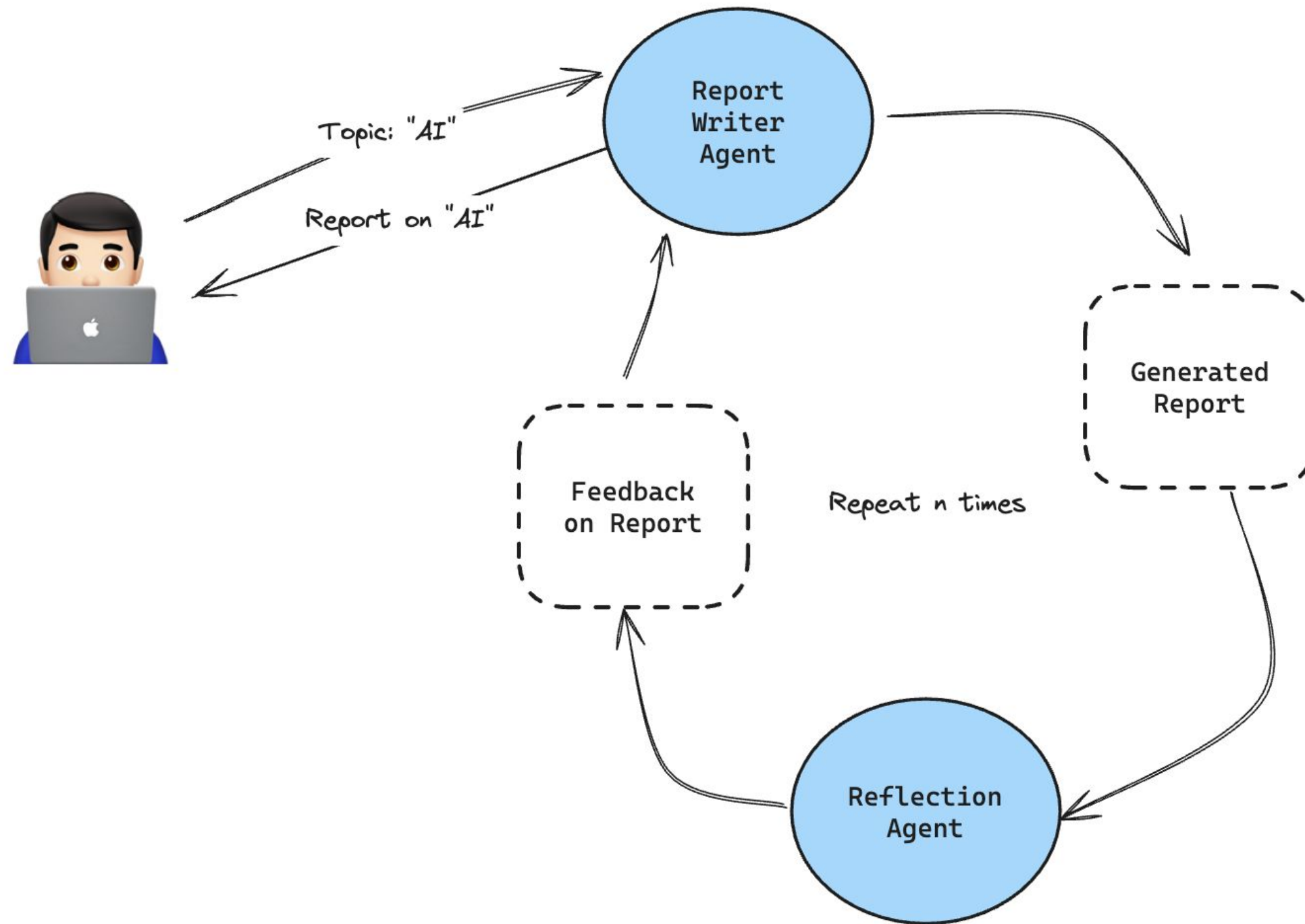




Introduction

- **Objective**
 - Review the *Reflection* architectural pattern for AI agents.
 - Learn how to implement reflection to enhance and improve agent outputs.
- **Core Idea**
 - Step 1: Initial task execution.
 - Step 2: Critique and evaluation.
 - Step 3: Output Improvement

Reflection for a Report Writing Agentic System



Reflection Workflow for Report Writing

Comparison with Non-Reflective Agents



- **Without Reflection:**
 - Agents produce outputs without assessing or improving based on feedback.
 - Higher likelihood of errors, inconsistencies, and lower quality results.
- **With Reflection:**
 - Continuous improvement loop ensures higher quality and more reliable outputs.
 - Enhances the agent's ability to learn from mistakes and adapt to feedback.
 - ***Caveat:*** *Can get stuck in infinite loops, so limit the number of cycles!*



3.10 Managing Conversation History in a Database



Conversation Threading Basics

- Allows thread level persistence for multi-turn conversations or interactions, allowing agents to remember context within a conversation
- Each thread is isolated (no shared context across threads)
- Useful way to manage sessions in multi session, multi user applications
- Core Components for threading in LangGraph:
 - **Thread ID:** Unique identifier for each conversation
 - **Checkpoint:** Persistence mechanism for conversation state

State Persistence Architecture in LangGraph

- **The Persistence Layer (Checkpointing)**
 - Saves conversation state after each interaction
 - Loads previous state when resuming
- **State Storage Options**
 - In-memory: Fast, lost on restart (development)
 - PostgreSQL/Redis: Production-grade, distributed systems
- **What Gets Saved**
 - Complete message history
 - Agent state variables
 - Metadata (timestamps, user info)

Thread Isolation and Context Management

- **Thread ID as Conversation Identifier**
 - Unique per conversation & user (e.g., user-123-session-1)
 - Different threads = completely isolated contexts
- **Session Management Scenarios**
 - **Single session:** One ongoing conversation
 - **Multi-session:** User can have multiple independent chats - akin to how you have separate chats in ChatGPT

How State Persists Over Time

- **Load:** Agent invoked with thread ID → retrieve latest state
- **Process:** Agent executes logic with full conversation context
- **Save:** New state snapshot (checkpoint) stored to database
- **Link:** New checkpoint references previous one (history chain)

O'REILLY®

3.11 Implementing Semantic Memory with Vector Stores



Semantic Memory

- **Core Concept**

- A long-term memory system that retrieves information by meaning
- Searches based on similarity to current context
- Enables cross-session knowledge persistence

- **Use Cases**

- Remembering user preferences across sessions
- Recalling relevant past solutions and strategies
- Building domain knowledge over time

Semantic Memory Architecture

- **Storage Layer**
 - Vector database (e.g., Postgres, Redis, MongoDB)
 - Stores memory items with vector embeddings
- **Retrieval Flow**
 - Current context is embedded using same model
 - Vector similarity search finds relevant memories
 - Top-k similar items are retrieved
 - Retrieved context enhances agent's prompt

Semantic Memory Architecture

- **Storage Workflow**
 - Important interactions are identified
 - Memory text is created with metadata
 - Embedding is generated and stored
 - Indexed for future similarity search

Memory Organization Strategies

- **Namespace Organization**

- Hierarchical structure: (user_id, "memories")
- Isolates memories per user or context
- Enables multi-tenant applications

- **Metadata Filtering**

- Store contextual information (user_id, timestamp, topic)
- Filter searches by metadata constraints
- Combine similarity with structured criteria

Integration Points

- **Retrieval Phase**

- Query memory store before generating response
- Use current context to form similarity search
- Retrieve top-k most relevant memories
- Inject retrieved context into agent's prompt

- **Storage Phase**

- Store important information after each interaction
- Extract key facts and preferences to persist
- Add metadata for future filtering
- Update or consolidate existing memories

Design Considerations & Maintenance

- **Design Decisions**
 - Balance retrieval cost vs. context quality
 - Determine optimal number of memories (k) to retrieve
 - Agent-controlled vs automatic memory management
 - Monitor latency impact on user experience
- **Memory Maintenance**
 - Update existing memories when information changes
 - Summarize old or redundant information
 - Delete outdated or irrelevant memories
 - Monitor storage growth and performance

Common Pitfall: Memory without Forgetting

Semantic memory grows forever, retrieval slows down, and ancient irrelevant memories surface - which may not be relevant to your user's current situation.

Implement memory decay, importance-based pruning, or explicit forgetting mechanisms.

O'REILLY®

3.12 Deploying Agents with FastAPI



From Local Experiments to Deployed Systems

- **Local Development is Great For**
 - Rapid prototyping and testing ideas
 - Debugging agent behavior
- **But**
 - Only you can use it (runs on your machine)
 - No way for a web app or mobile app to call your agent
- **Deployment Enables:**
 - **Accessibility:** Anyone (or any system) can interact with your agent via HTTP
 - **Integration:** Web apps, mobile apps, Slack bots, other services can all call your agent
 - **Scalability:** Run on powerful cloud servers, serve multiple users
 - **Persistence:** Agent stays running 24/7, not tied to your laptop being open

FastAPI for Deploying Agents

- FastAPI is a modern Python web framework ideal for AI applications.
- **Features**
 - **Async Support:** Handle multiple agent requests concurrently
 - **Automatic Docs:** OpenAPI/Swagger UI generated from code
 - **Type Hints:** Request/response validation with Pydantic
 - **Simple Syntax:** Minimal boilerplate to expose endpoints
- **Agent Framework Agnostic:** FastAPI is just the HTTP layer - it works with any agent framework (LangGraph, CrewAI, custom agents, etc.)

FastAPI Core Concepts

- **App Instance:** The FastAPI application object
- **Routes/Endpoints:** URL paths that handle requests (e.g., /chat, /invoke)
- **HTTP Methods:** GET (retrieve), POST (send data), etc.
- **Request Body:** Data sent by the client (typically JSON)
- **Response:** Data returned to the client

Exposing an Agent as an Endpoint

- **The Pattern**

- Create a FastAPI app instance
- Define a Pydantic model for request validation
- Create a POST endpoint that accepts user input
- Inside the endpoint, invoke your agent
- Return the agent's response

- **What Happens at Runtime**

- FastAPI handles HTTP parsing, validation, serialization
- Your code focuses purely on agent logic
- Errors are automatically converted to proper HTTP responses

Running Your FastAPI Server

- **Development vs Production**
 - **Development:** `uvicorn app:app --reload` - Auto-reloads on code changes
 - **Production:** `uvicorn app:app --workers 4` - Multiple worker processes for **concurrency**
- **Key uvicorn Options**
 - `--host 0.0.0.0`: Accept connections from any IP (needed for deployment)
 - `--port 8000`: Which port to listen on
 - `--reload`: Restart on file changes (dev only)
 - `--workers N`: Run N parallel processes (production)
- **Testing Your Endpoint**
 - Built-in docs at `http://localhost:8000/docs`
 - Use curl, Postman, or any HTTP client

Basic Deployment Architecture



O'REILLY®